

Final Research Project

Advanced Transportation Systems Modeling: Flow-Through Tensor, Data Cleaning & Fusion, NGSIM Calibration, and Traffic Simulation

Transportation Systems Optimization Course

Fall 2025

1 Introduction

This final research project integrates modern AI/ML concepts (tensors, propagation, attention mechanisms) with traffic flow modeling, data pipelines, and simulation frameworks. The project is designed around the **A–E Architecture Alphabet framework** for systematic transportation system analysis, connecting classical optimization structures with emerging deep learning methods.

Each team should select **one primary component** while making explicit connections to other components where appropriate. All teams must demonstrate reproducible results with complete datasets and code repositories.

2 The Architecture Alphabet (A–E): Theoretical Foundation

2.1 Overview

The Architecture Alphabet provides a unified taxonomy that classifies transportation modeling and optimization approaches into five complementary families. This framework clarifies how our project components relate to both classical optimization structures and modern deep learning methods.

2.2 A: Arc-based and Arc-to-Path Architectures

Arc-based (ADN) architectures represent feasible movements explicitly as time-space nodes and arcs. Examples include dynamic time-space networks for trains and vehicles, link-time formulations of CTM/queue-based DTA, and phase-based signal control models. Formally, the state and decisions are embedded in a sparse graph:

$$\mathcal{G}^A = (\mathcal{N}^{\text{time-space}}, \mathcal{A}^{\text{time-space}}) \quad (1)$$

on which shortest path, multi-commodity flow, and scheduling problems are solved. This class encompasses the classical time-space network literature and remains the physical backbone for most transportation models.

2.3 B: Backpropagation-based Deep Networks

Class B denotes the *computational graph* layer: CNNs, RNNs, GNNs, Transformers, and diffusion models trained by gradient-based methods. A model in class B is a differentiable mapping:

$$y = \Phi_{\theta}(x) \quad (2)$$

with parameters θ learned from data. Crucially, B-type architectures are *operators*, not problem structures—they provide powerful function approximation on top of whatever structural architecture (A, C, D) and coordinate system (E) we choose. The transformer architecture computes attention as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

This operation is inherently rank-constrained: outputs lie in the span of value vectors V , making transformers natural candidates for low-rank latent space operations.

2.4 C: Column and Path-based Architectures

Column-based architectures use *paths* or *composite plans* as basic variables: an entire path, train run, duty roster, or service plan is treated as a single column. A master problem imposes capacity and resource constraints, while column generation iteratively enriches the column set:

$$x = \sum_{k \in \mathcal{K}} \lambda_k e_k, \quad \lambda_k \geq 0 \quad (4)$$

This class includes path-based traffic assignment, railway line planning, crew scheduling, and bundle methods. Path flows naturally align with origin-destination demand patterns and provide interpretable decision variables.

2.5 D: Dualization-based Cross-layer Architectures

Class D couples multiple layers or modules through *dual variables*, shadow prices, and Lagrange multipliers. Examples include Benders decomposition, Lagrangian relaxation, ADMM, and cross-layer optimization in multi-sector infrastructure systems. Different layers (demand, supply, control, communication) solve subproblems given dual signals:

$$\min_x \max_{\lambda} L(x, \lambda) = f(x) + \lambda^{\top} g(x) \quad (5)$$

with dual variables λ mediating cross-layer interactions. This structure naturally extends to multi-modal and multi-sector systems where different infrastructure networks must coordinate.

2.6 E: Eigen, Energy, Spectrum, and Tensor Views

Class E provides a spectrum- and tensor-based coordinate system for traffic states, demand-supply imbalance, and queues. Instead of working in raw variable space, we construct multiway tensors:

$$\mathcal{X}(\text{link}, \text{time}, \text{OD}, \text{class}, \text{layer}) \quad (6)$$

and apply low-rank decompositions (SVD, CP/Tucker) to obtain physically meaningful latent modes. These modes serve as an “eigen-coordinate system” for expressing demand, supply, and queue dynamics. This view unifies tensor methods in traffic data imputation, multi-modal demand analysis, and supply chain optimization.

2.7 How the Alphabet Fits Together

The key insight is that A, C, and D encode *problem structure*—time-space graphs, column spaces, and dual couplings—while E provides a *spectral coordinate system* on top of these structures. Class B (deep networks) then operates *within* these structured, spectral architectures as flexible parametric approximators, not as replacements for physical models. Our tensor-transformer framework explicitly combines all five classes: arc-based physical constraints (A), transformer attention operators (B), path-based demand representation (C), multi-layer optimization (D), and low-rank tensor coordinates (E).

3 Project Components

3.1 Part 1. Flow-Through Tensor with DTALite

Contact: Henan – flow-through tensor lead

3.1.1 Theoretical Foundation

Based on Zhou et al.’s “Flow-through tensors: A unified computational graph architecture for multi-layer transportation network optimization” framework.

3.1.2 Architecture Alphabet Connection

This component directly implements:

- **Class A:** Arc-based time-space network formulation in DTALite
- **Class C:** Path-based flow representation through OD demand
- **Class E:** Tensor decomposition for spectral coordinates

3.1.3 Objectives

- Implement DTALite to generate flow-through tensors for test networks
- Construct multi-dimensional tensor representations: $\mathcal{X}(\text{OD}, \text{path}, \text{time})$ or $\mathcal{X}(\text{link}, \text{time}, \text{scenario})$
- Perform tensor-based calibration of link volumes using flow propagation algorithms
- Apply SVD or Tucker decomposition for dimensionality reduction
- Demonstrate how tensor structures encode propagation “layer by layer”

3.1.4 Expected Work

1. Run DTALite for the provided network across multiple time periods
2. Construct and visualize tensor structures showing flow propagation layer-by-layer
3. Implement SVD or Tucker decomposition:

$$\mathcal{X} \approx \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \times_3 U^{(3)} \quad (7)$$

where $\mathcal{G}$ is the core tensor and $U^{(i)}$ are factor matrices

4. Calibrate model parameters using compressed tensor representations
5. Document computational efficiency gains (e.g., reduction from millions to hundreds of variables) and solution quality preservation
6. Compare compressed vs. full-rank solutions

3.1.5 Key References

- Zhou et al., “Flow-through tensors: A unified computational graph architecture for multi-layer transportation network optimization”
- DTALite documentation and tutorials on GitHub
- Supplementary materials: <https://drive.google.com/file/d/10LEMEal-cAYkFLCzViT0JadQGIPt8jW1/view>

3.2 Part 2. Data Cleaning

Contact: Yajun – data cleaning lead

3.2.1 Theoretical Foundation

Based on USDOT JPO CodeHub data cleaning and fusion tool, plus FHWA guidelines.

3.2.2 Architecture Alphabet Connection

- **Preprocessing for all classes:** Clean data feeds into A (arc-based models), C (path flows), and E (tensor construction)
- **Class B preparation:** Quality data is essential for training deep learning models

3.2.3 Objectives

- Master FHWA data cleaning packages for multiple data sources (loop detectors, CV/trajjectory, weather, incidents)
- Develop robust pipelines for handling missing data, outliers, and sensor errors
- Establish data quality metrics and validation procedures
- Create reproducible cleaning workflows

3.2.4 Expected Work

1. Study FHWA GitHub repositories:
<https://github.com/usdot-jpo-codehub/data-cleaning-and-fusion-tool>
2. Review FHWA guidelines document (provided on Canvas)
3. Implement cleaning pipeline on sample datasets:
 - Loop detector data (volume, occupancy, speed)

- GPS/CV trajectory data
 - Weather and incident data
4. Perform statistical analysis:
 - Completeness rates by sensor and time period
 - Outlier detection using statistical tests (e.g., IQR, z-scores)
 - Temporal consistency checks
 - Spatial consistency across adjacent sensors
 5. Document data quality metrics before and after cleaning
 6. Create reproducible cleaning scripts with comprehensive documentation

3.2.5 Key References

- USDOT JPO CodeHub: data-cleaning-and-fusion-tool repository
- FHWA Data Cleaning Guidelines report
- Google Drive supplementary materials

3.3 Part 3. Data Fusion

Contact: Yajun – data fusion lead

3.3.1 Theoretical Foundation

Multi-source data fusion using FHWA tools and methodologies for enhanced accuracy and usability of traffic models.

3.3.2 Architecture Alphabet Connection

- **Class E enabler:** Fused data populates tensor dimensions consistently
- **Cross-architecture integration:** Unified datasets support arc-based (A), path-based (C), and learning-based (B) methods

3.3.3 Objectives

- Integrate heterogeneous data sources (loop detectors + trajectory + weather + incidents)
- Create consistent, high-fidelity datasets for calibration and validation
- Enable high-fidelity bottleneck identification and route choice analysis
- Support model calibration across different architectures

3.3.4 Expected Work

1. Extend Part 2 cleaning work to multi-source fusion
2. Implement spatial-temporal alignment algorithms:
 - Map trajectories to network links
 - Synchronize loop detector and probe data timestamps
 - Correlate weather/incident data with traffic observations
3. Demonstrate concrete fusion examples:
 - Loop detector + GPS trajectory data for speed-flow relationships
 - Speed profiles + weather conditions for capacity analysis
 - Incident data + traffic flow impacts for bottleneck identification
4. Validate fused datasets:
 - Cross-validation between data sources
 - Consistency checks (e.g., flow conservation)
 - Comparison with ground truth where available
5. Prepare fusion outputs for tensor calibration (Part 1) or NGSIM work (Part 4)
6. Document fusion methodology, algorithms, and quality assessment metrics

3.3.5 Key References

- USDOT JPO data fusion tool repository
- FHWA fusion guidelines and best practices
- Provided tutorials and sample datasets

3.4 Part 4. NGSIM Trajectory-Based Flow Model Calibration

Contact: Prof. Simon Zhou – NGSIM setup and calibration methods

3.4.1 Theoretical Foundation

Based on trajectory data analysis methods including:

- Treiber & Helbing: “Method for investigating intradriver heterogeneity using vehicle trajectory data: A Dynamic Time Warping approach”
- Zhou et al.: “Trajectory data-based traffic flow studies: A revisit”

3.4.2 Architecture Alphabet Connection

- **Class A:** Microscopic arc-based dynamics (car-following, lane-changing)
- **Class B:** DTW as a differentiable distance metric for gradient-based calibration
- **Class E:** Trajectory aggregation into space-time tensor representations

3.4.3 Objectives

- Process NGSIM trajectory datasets into cell-based traffic state variables
- Calibrate microscopic traffic flow models using Dynamic Time Warping (DTW)
- Investigate intradriver heterogeneity through trajectory pattern analysis
- Validate calibrated models on hold-out data

3.4.4 Expected Work

1. Use NGSIM visualization and trajectory converter tools
2. Define cell structure (e.g., 50m cells) and aggregate trajectories into:
 - Cell-level density $\rho(x, t)$, flow $q(x, t)$, speed $v(x, t)$ time series
 - Acceleration profiles: $a(t) = \frac{dv}{dt}$
 - Gap distributions: $s(t)$ between successive vehicles

3. Implement DTW-based calibration framework:

$$\text{DTW}(X, Y) = \min_{\pi} \sum_{(i,j) \in \pi} d(x_i, y_j) \quad (8)$$

where π is an alignment path between observed (X) and simulated (Y) trajectories

4. Compare simulated vs. observed trajectories using DTW distance
5. Estimate car-following model parameters (e.g., IDM, Gipps):
 - Desired velocity v_0
 - Safe time headway T
 - Maximum acceleration/deceleration a, b
6. Calibrate fundamental diagram relationships: $q = \rho \cdot v(\rho)$
7. Analyze intradriver heterogeneity:
 - Cluster vehicles by DTW distance in parameter space
 - Quantify distribution of behavioral parameters
8. Validate calibrated models on hold-out trajectory segments
9. Document calibration convergence and parameter sensitivity

3.4.5 Key References

- Taylor & Zhou, “Method for investigating intradriver heterogeneity using vehicle trajectory data: A Dynamic Time Warping approach”
- Zhou et al., “Trajectory data-based traffic flow studies: A revisit”
- NGSIM dataset and converter tools (provided on Canvas/GitHub)

3.5 Part 5. Traffic Simulation and Transformer-Style Architecture

Contact: Mohammed – traffic simulation / transformer architecture lead

3.5.1 Theoretical Foundation

Based on Lu and Zhou’s “Virtual track networks: A hierarchical modeling framework and open-source tools for simplified and efficient connected and automated mobility (CAM) system design based on general modeling network specification (GMNS)” with transformer/attention mechanism concepts.

3.5.2 Architecture Alphabet Connection

This component synthesizes all five classes:

- **Class A:** GMNS-based arc representation of virtual track networks
- **Class B:** Transformer attention mechanism for state updates
- **Class C:** Path-based routing decisions through virtual tracks
- **Class D:** Multi-layer coordination (physical, virtual, control layers)
- **Class E:** Attention weights as spectral decomposition of spatial dependencies

3.5.3 Objectives

- Prototype transformer-style architecture for traffic state prediction
- Map Q-K-V attention mechanisms to traffic flow dynamics
- Demonstrate interpretability through attention weight visualization
- Compare with traditional simulation approaches (CTM, kinematic waves)

3.5.4 Expected Work

1. Load node-link network using GMNS format (from Virtual Track Networks framework):

- node.csv: intersection/junction nodes
- link.csv: road segments with capacity, free-flow speed
- movement.csv (optional): turning movements

2. Design attention-based state representation:

- **Query (Q):** Current traffic state at target location (density, queue length)

$$Q_i = W_Q \cdot [\rho_i, q_i, v_i]^\top \quad (9)$$

- **Key (K):** Historical/spatial features from upstream/downstream links

$$K_j = W_K \cdot [\rho_j, q_j, v_j, \text{distance}_{ij}]^\top \quad (10)$$

- **Value (V):** Flow propagation information and control signals

$$V_j = W_V \cdot [\Delta q_j, \Delta \rho_j, \text{signal}_j]^\top \quad (11)$$

3. Implement simplified transformer layer for traffic state updates:

$$\text{Attention}_{ij} = \text{softmax} \left(\frac{Q_i K_j^\top}{\sqrt{d_k}} \right) \quad (12)$$

$$\rho_i^{t+1} = \rho_i^t + \sum_j \text{Attention}_{ij} \cdot V_j \quad (13)$$

4. Visualize attention weights to show spatial-temporal dependencies:

- Heatmaps of Attention_{ij} matrix
- Network plots with edge weights proportional to attention
- Time evolution of attention patterns

5. Compare with traditional simulation approaches:

- Cell Transmission Model (CTM): $\rho_i^{t+1} = \rho_i^t + \Delta t(q_{in} - q_{out})$
- Kinematic wave model
- Queue-based methods

6. Document conceptual architecture:

- Architecture diagrams showing Q-K-V flow
- Code demonstrations with clear comments
- Interpretability analysis of learned attention patterns

7. Discuss connections to physics:

- How attention weights relate to wave propagation
- Physical constraints (flow conservation, capacity)
- Stability and causality properties

3.5.5 Key References

- Zhou et al., “Virtual track networks: A hierarchical modeling framework and open-source tools for simplified and efficient connected and automated mobility (CAM) system design based on general modeling network specification (GMNS)”
- GMNS specification and tools: <https://github.com/asu-trans-ai-lab/GMNS>
- Vaswani et al., “Attention is all you need” (for transformer concepts)

4 Deliverables (Required for Each Team)

4.1 1. Comprehensive Report (12–15 pages)

4.1.1 Introduction & Motivation (1–2 pages)

- Problem statement and research gap
- Literature review incorporating provided references
- Research objectives tied to Architecture Alphabet framework
- Contribution and novelty

4.1.2 Theoretical Framework (4–6 pages)

- Detailed explanation of Architecture Alphabet classes (A–E) relevant to your component
- Mathematical formulations with clear notation
- Key algorithms with pseudocode where appropriate
- Connections between different architecture classes
- How your approach integrates multiple classes

4.1.3 Data & Implementation (2–3 pages)

- Dataset descriptions with summary statistics:
 - Number of observations, spatial/temporal coverage
 - Data quality metrics
 - Preprocessing steps
- Software tools and computational environment:
 - Programming languages (Python, C++, etc.)
 - Key libraries (NumPy, TensorFlow/PyTorch, NetworkX, etc.)
 - Hardware specifications
- Implementation details:
 - Algorithm complexity analysis
 - Data structures used
 - Optimization techniques applied

4.1.4 Results & Analysis (4–5 pages)

- Quantitative results with professional-quality figures and tables:
 - Convergence plots
 - Performance metrics (RMSE, MAPE, R^2 , etc.)
 - Comparison with baseline methods
- Validation metrics and statistical tests
- Sensitivity analysis:
 - Parameter variations
 - Robustness checks
 - Uncertainty quantification
- Computational efficiency analysis:
 - Runtime comparisons
 - Memory usage
 - Scalability analysis
- Interpretation of results in context of Architecture Alphabet

4.1.5 Lessons Learned & Future Work (1–2 pages)

- Technical challenges encountered and solutions developed
- Computational insights and best practices
- Limitations of current approach
- Extensions and research directions:
 - How to integrate with other project components
 - Potential applications to real-world problems
 - Open research questions
- Recommendations for practitioners

4.2 2. Reproducible Code Repository

Create a **complete GitHub repository** including:

- **Source code:**
 - Clean, well-commented, modular code
 - Follow PEP 8 (Python) or equivalent style guides
 - Include docstrings for all functions/classes
- **Jupyter notebooks:**

- Step-by-step execution with explanations
- Inline visualization of results
- Clear markdown cells explaining each section
- **README.md:**
 - Project overview and objectives
 - Installation instructions (OS-specific if needed)
 - Quick start guide
 - Detailed usage examples
 - Contact information
- **Data:**
 - Sample datasets (if small enough for Git)
 - Download scripts for larger datasets
 - Data dictionary explaining all variables
 - Preprocessing scripts
- **Environment:**
 - requirements.txt (Python) or environment.yml (Conda)
 - Dockerfile (optional but recommended)
 - Version information for all dependencies
- **Validation:**
 - Example outputs for verification
 - Unit tests for critical functions
 - Validation scripts with expected results
- **Documentation:**
 - API documentation (Sphinx, pdoc, etc.)
 - Tutorial notebooks
 - FAQ section

4.3 3. Extended Presentation (15 minutes + 5 min Q&A)

Structure:

1. **Research motivation and objectives (2 min)**
 - Problem context and significance
 - Research gap
 - Key research questions
2. **Theoretical framework and Architecture Alphabet connections (3 min)**

- Which classes (A, B, C, D, E) are involved
- How they interact in your approach
- Key mathematical concepts

3. Methodology and implementation (4 min)

- Algorithm overview with flowchart/diagram
- Data pipeline
- Computational architecture
- Implementation highlights

4. Key results with visualizations (4 min)

- Main findings with high-quality figures
- Performance metrics
- Comparison with baselines
- Surprising or interesting discoveries

5. Lessons learned and future directions (2 min)

- Key takeaways
- Practical implications
- Future research opportunities
- Connections to other project components

Presentation guidelines:

- Use clear, professional slides (avoid clutter)
- Include high-quality figures and diagrams
- Practice timing to stay within 15 minutes
- Prepare for technical questions
- Bring backup slides for anticipated questions
- Make code and data repository accessible (QR code on final slide)

5 Evaluation Criteria

5.1 Technical Depth (30%)

- Mastery of relevant methods and algorithms
- Correct implementation and validation
- Theoretical understanding demonstrated
- Appropriate use of Architecture Alphabet framework
- Mathematical rigor and precision

5.2 Reproducibility (25%)

- Code quality, organization, and documentation
- Clear instructions for reproduction
- Dataset availability and accessibility
- Successful execution by independent reviewer
- Version control and repository management

5.3 Integration (20%)

- Explicit connections to A–E Architecture Alphabet
- Links to other project parts (data flow, methods, results)
- Synthesis of multiple architecture classes
- Cross-component validation where applicable
- Systems thinking and holistic view

5.4 Innovation (15%)

- Novel insights or approaches
- Creative problem-solving
- Extensions beyond provided materials
- Original contributions to the field
- Potential for publication or real-world application

5.5 Communication (10%)

- Report clarity, organization, and writing quality
- Presentation effectiveness and engagement
- Quality of visualizations and figures
- Ability to explain complex concepts clearly
- Professional polish and attention to detail

6 Resources and Materials

All GitHub links, training materials, reference papers, and datasets are available on Canvas under the “Final Project” module.

6.1 Key GitHub Repositories

- DTALite: <https://github.com/asu-trans-ai-lab/DTALite>
- GMNS: <https://github.com/asu-trans-ai-lab/GMNS>
- USDOT Data Cleaning: <https://github.com/usdot-jpo-codehub/data-cleaning-and-fusion-tool>
- Virtual Track Networks: (link on Canvas)

6.2 Required References

- Zhou et al., “Flow-through tensors: A unified computational graph architecture for multi-layer transportation network optimization”
- Zhou et al., “Trajectory data-based traffic flow studies: A revisit”
- Zhou et al., “Virtual track networks: A hierarchical modeling framework...”
- Treiber & Helbing, “Method for investigating intradriver heterogeneity using vehicle trajectory data: A Dynamic Time Warping approach”
- FHWA Data Cleaning and Fusion Guidelines
- Vaswani et al., “Attention is all you need”

6.3 Supplementary Materials

- Google Drive folder: <https://drive.google.com/file/d/10LEMEal-cAYkFLCzViTOJadQGIPt8jW1/view>
- NGSIM dataset and visualization tools
- Sample networks and test cases
- Tutorial notebooks and example code

7 Submission

7.1 What to Submit

1. PDF report (via Canvas)
2. GitHub repository URL (public or private with instructor access)
3. Presentation slides (PDF format)
4. (Optional) Supplementary materials: demo video, additional visualizations, etc.

7.2 Submission Deadline

- **Report, code, and slides:** Dec 2nd 12PM
- **Presentations:** Dec 2nd, and 3rd

8 Final Remarks

This is a substantial research project requiring significant effort and coordination. The goal is to produce **publication-quality work** that advances the state of practice in transportation systems modeling by integrating classical optimization structures (A–E Architecture Alphabet) with modern deep learning methods.

Key success factors:

- Start early and maintain consistent progress
- Communicate regularly with your component lead and team
- Leverage provided tools, frameworks, and examples
- Focus on reproducibility from day one
- Connect your work explicitly to the Architecture Alphabet framework
- Document everything as you go
- Seek feedback early and often

We look forward to seeing innovative solutions that bridge classical transportation modeling with cutting-edge AI/ML techniques!

Questions? Contact Prof. Simon Zhou or your designated component lead.